Практикум 2.7. Дифференцирование функций многих переменных. Формула Тейлора

Упражнение 1.

а) Вычислите частные производные первого и второго порядка функции $z(x, y) = \cos(3x + y^2)$.

б) Найдите градиент функции $f(x, y, z) = 2x^3 y + x - z$ в точке $(1, 2, -3)$.

а)

```
x, y = sp.Symbol('x'), sp.Symbol('y')
z = sp.cos(3 * x + y**2)

dzdx = sp.diff(z, x, 1)
dzdy = sp.diff(z, y, 1)
d2zdxdy = sp.diff(dzdx, y, 1)
d2zdydx = sp.diff(dzdy, x, 1)
d2zdx2 = sp.diff(z, x, 2)
d2zdy2 = sp.diff(z, y, 2)
print('dzdx =', dzdx)
print('dzdy =', dzdy)
print('d2zdxdy =', d2zdxdy)
print('d2zdydx =', d2zdydx)
print('d2zdx2 =', d2zdx2)
print('d2zdy2 =', d2zdy2)
```

```
dzdx = -3*sin(3*x + y**2)
dzdy = -2*y*sin(3*x + y**2)
d2zdxdy = -6*y*cos(3*x + y**2)
d2zdydx = -6*y*cos(3*x + y**2)
d2zdx2 = -9*cos(3*x + y**2)
d2zdy2 = -2*(2*y**2*cos(3*x + y**2) + sin(3*x + y**2))
```

б)

```
x = sp.Symbol('x')
y = sp.Symbol('y')
z = sp.Symbol('z')
f = 2 * x**3 * y + x - z

grad_x = sp.diff(f, x).subs([(x, 1), (y, 2), (z, -3)])
grad_y = sp.diff(f, y).subs([(x, 1), (y, 2), (z, -3)])
grad_z = sp.diff(f, z).subs([(x, 1), (y, 2), (z, -3)])

print('grad{f}|(1, 2, -3) = (', grad_x, grad_y, grad_z, ')')
```

```
grad{f}|(1, 2, -3) = ( 13 2 -1 )
```

Упражнение 2.

Вычислите якобиан перехода от декартовой системы координат к цилиндрической (переход осуществляется по формулам: $x = r\cos\varphi$, $y = r\sin\varphi$, $z = z$).

```
r, t = sp.symbols('r t')
x = r * sp.cos(t)
y = r * sp.sin(t)
```

```
z = z
A = sp.Matrix([[x, y, z]])
B = sp.Matrix([[sp.diff(A, r)], [sp.diff(A, t)], [sp.diff(A, z)]])
B
```

$$\begin{bmatrix} \cos(t) & \sin(t) & 0 \\ -r\sin(t) & r\cos(t) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

```
sp.pprint(sp.simplify(B.det()))
```

```
r
```

Упражнение 3.

а) Найдите первый дифференциал dz функции $z = xy^2 + 2y - x^2$ в точке $(2,-1)$, если $\Delta x = 0{,}1$, $\Delta y = -0{,}2$.

а)

```
x = sp.Symbol('x')
y = sp.Symbol('y')
dx = 0.1
dy = -0.2
z = x * y**2 + 2 * y - x**2
dz = sp.diff(z, x).subs([(x, 2), (y, -1)]) * dx
dz += sp.diff(z, y).subs([(x, 2), (y, -1)]) * dy
print(dz)
```

```
0.100000000000000
```

б) Создайте **функцию**, вычисляющую первый дифференциал функции $f(x,y)$ в точке (x_0, y_0) при приращениях Δx, Δy. В число входных параметров включите функцию f, ее аргументы x, y и их приращения Δx, Δy, заданные в гументов. В число выходных параметров включите символьное выражение первого дифференциала в точке (x_0, y_0) и его числовое значение при заданных приращениях аргументов. Протестируйте **функцию**, используя данные пункта а).

б)

```
dx = sp.Symbol('dx')
dy = sp.Symbol('dy')
x = sp.Symbol('x')
y = sp.Symbol('y')

def df(f, x, y, dx, dy, x0, y0, dx0, dy0):
    df = sp.diff(f, x) * dx + sp.diff(f, y) * dy
    df_0 = df.subs([(x, x0), (y, y0), (dx, dx0), (dy, dy0)])
    return df, df_0

dx0 = 0.1
dy0 = -0.2
```

```
z = x * y**2 + 2 * y - x**2
df(z, x, y, dx, dy, 2, -1, dx0, dy0)
```

```
(dx*(-2*x + y**2) + dy*(2*x*y + 2), 0.100000000000000)
```

Упражнение 4.

а) Найдите второй дифференциал d^2z функции $z = xy^2 + 2y - x^2$ в точке $(2,-1)$, если $\Delta x = 0{,}1$, $\Delta y = -0{,}2$.

б) Создайте **функцию**, вычисляющую второй дифференциал функции $f(x,y)$ в точке (x_0, y_0) при приращениях Δx, Δy. В число входных параметров включите функцию f, ее аргументы x, y и их приращения Δx, Δy, заданные в символьном виде, координаты точки (x_0, y_0) и числовые значения приращений аргументов. В число выходных параметров включите символьное выражение второго дифференциала в точке (x_0, y_0) и его числовое значение при указанных приращениях аргументов. Протестируйте **функцию**, используя данные пункта а).

а)

```
x = sp.Symbol('x')
y = sp.Symbol('y')
dx = 0.1
dy = -0.2
z = x * y**2 + 2 * y - x**2
d2z = sp.diff(z, x, 2).subs([(x, 2), (y, -1)]) * dx**2
d2z += sp.diff(z, y, 2).subs([(x, 2), (y, -1)]) * dy**2
d2z += 2 * sp.diff(sp.diff(z, x), y).subs([(x, 2), (y, -1)]) * dx * dy
print(d2z)
```

```
0.220000000000000
```

б)

```
dx = sp.Symbol('dx')
dy = sp.Symbol('dy')
x = sp.Symbol('x')
y = sp.Symbol('y')

def d2f(f, x, y, dx, dy, x0, y0, dx0, dy0):
    d2f = sp.diff(f, x, 2) * dx**2 + sp.diff(f, y, 2) * dy**2
    d2f += 2 * sp.diff(sp.diff(f, x), y) * dx * dy
    d2f_0 = d2f.subs([(x, x0), (y, y0), (dx, dx0), (dy, dy0)])
    return d2f, d2f_0

dx0 = 0.1
dy0 = -0.2
z = x * y**2 + 2 * y - x**2
d2f(z, x, y, dx, dy, 2, -1, dx0, dy0)
```

```
(-2*dx**2 + 4*dx*dy*y + 2*dy**2*x, 0.220000000000000)
```

а) Создайте **функцию**, раскладывающую функцию $f(x,y)$ в точке (x_0,y_0) в ряд Тейлора до членов 1-го порядка включительно. В число входных параметров включите функцию f, ее аргументы x,y и их приращения Δx, Δy, заданные в символьном виде, координаты точки (x_0,y_0) и числовые значения приращений аргументов. В число выходных параметров включите символьное разложение $p_1(x,y)$ функции $f(x,y)$ по формуле Тейлора в точке (x_0,y_0) до членов 1-го порядка включительно, записанное через произвольные значения аргументов функции, а также приближенное значение функции $f(x,y)$ в точке $x=x_0+\Delta x$, $y=y_0+\Delta y$ (значение $p_1(x_0+\Delta x, y_0+\Delta y)$) при указанных значениях x_0, y_0, dx, dy.

б) Используйте **функцию** из п. а) для вычисления приближенного значения функции $z=f(x,y)$ в точке $(x_0;y_0)$. Сравните полученный результат с точным значением этой функции в указанной точке.

в) Постройте в одной системе координат в области $x\in[a;b]$, $y\in[c,d]$ поверхности $z=f(x,y)$ и $z=p_1(x,y)$.

$f(x,y)$	(x_0,y_0)	$[a,b]$	$[c,d]$
$f(x,y)=(2x^2+y^2)^2$	$(0,8;2,2)$	$[-6;6]$	$[-6;6]$

а)

```
dx = sp.Symbol('dx')
dy = sp.Symbol('dy')
x = sp.Symbol('x')
y = sp.Symbol('y')

def taylor1(f, x, y, dx, dy, x0, y0, dx0, dy0):
    p1_symb = f.subs([(x, x0), (y, y0)]) + df(f, x, y, dx, dy, x0, y0, dx0, dy0)[0]
    p1 = f.subs([(x, x0), (y, y0)])
    p1 += df(f, x, y, dx, dy, x0, y0, dx0, dy0)[1]
    return p1_symb, p1
```

б)

```
dx = sp.Symbol('dx')
dy = sp.Symbol('dy')
x = sp.Symbol('x')
y = sp.Symbol('y')
f = (2 * x**2 + y**2)**2

taylor1(f, x, y, dx, dy, 1, 2, -0.2, 0.2)
```

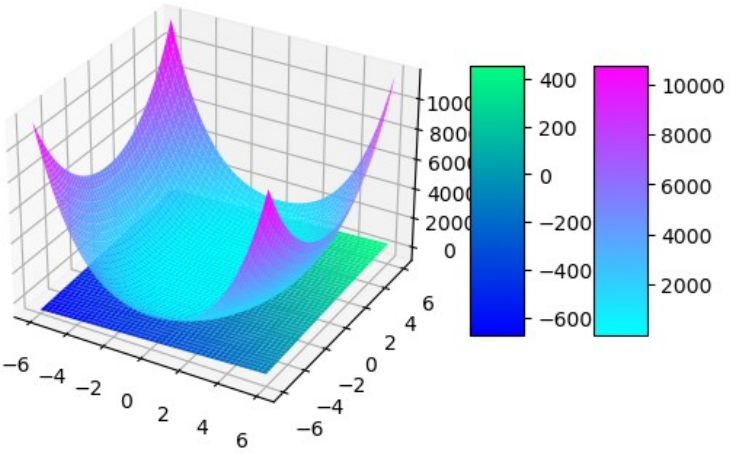
```
(8*dx*x*(2*x**2 + y**2) + 4*dy*y*(2*x**2 + y**2) + 36, 36)
```

```
(2 * 0.8**2 + 2.2**2)**2
```

```
37.454400000000014
```

в)

```
from matplotlib import cm

x0 = 1
y0 = 2

fig = plt.figure()
xs = np.linspace(-6, 6, 50)
ys = np.linspace(-6, 6, 50)

X, Y = np.meshgrid(xs, ys)
Z = (2 * X**2 + Y**2)**2
p1 = taylor1(f, x, y, dx, dy, 1, 2, -0.2, 0.2)[0]
ZZ = f.subs([(x, 1), (y, 2)]) + (X - 1) * 8 * x0 * (2 * x0**2 + y0**2)
ZZ += (Y - 2) * 4 * y0 * (2 * x0**2 + y0**2)

fig, ax = plt.subplots(subplot_kw={"projection": "3d"})
surf = ax.plot_surface(X, Y, Z, cmap=cm.cool, linewidth=0)
surf1 = ax.plot_surface(X, Y, ZZ, cmap=cm.winter, linewidth=0)
fig.colorbar(surf, shrink=0.5, aspect=5)
fig.colorbar(surf1, shrink=0.5, aspect=5)
plt.show()
```



Упражнение 6.

а) Создайте **функцию**, раскладывающую функцию $f(x,y)$ в точке (x_0,y_0) по формуле Тейлора до членов 2-го прядка включительно. В число входных параметров включите функцию f, ее аргументы x,y и их приращения Δx, Δy, заданные в символьном виде, координаты точки (x_0,y_0) и числовые значения приращений аргументов. В число выходных параметров включите символьное разложение $p_2(x,y)$ функции $f(x,y)$ по формуле Тейлора в точке (x_0,y_0) до членов 2-го порядка включительно, записанное через произвольные значения аргументов функции, а также приближенное значение функции $f(x,y)$ в точке $x=x_0+\Delta x$, $y=y_0+\Delta y$ (значение $p_2(x_0+\Delta x, y_0+\Delta y)$) при указанных значениях x_0, y_0, dx, dy.

б) Используйте **функцию** из п. а) для вычисления приближенного значения функции $z = f(x, y)$ в точке $(x_0; y_0)$. Сравните полученный результат с точным значением этой функции в указанной точке и с ее приближенным значением, полученным по формуле Тейлора до членов 1-го порядка.

в) Постройте в одной системе координат в области, $x \in [a;b]$, $y \in [c,d]$ поверхности $z = f(x, y)$, $z = p_1(x, y)$ и $z = p_2(x, y)$.

а)

```
dx = sp.Symbol('dx')
dy = sp.Symbol('dy')
x = sp.Symbol('x')
y = sp.Symbol('y')

def taylor2(f, x, y, dx, dy, x0, y0, dx0, dy0):
    p2_symb = f.subs([(x, x0), (y, y0)]) + df(f, x, y, dx, dy, x0, y0, dx0, dy0)[0]
    p2_symb += d2f(f, x, y, dx, dy, x0, y0, dx0, dy0)[0] / 2
    p2 = f.subs([(x, x0), (y, y0)])
    p2 += df(f, x, y, dx, dy, x0, y0, dx0, dy0)[1]
    p2 += d2f(f, x, y, dx, dy, x0, y0, dx0, dy0)[1] / 2
    return p2_symb, p2
```

б)

```
dx = sp.Symbol('dx')
dy = sp.Symbol('dy')
x = sp.Symbol('x')
y = sp.Symbol('y')
f = (2 * x**2 + y**2)**2

print(taylor2(f, x, y, dx, dy, 1, 2, -0.2, 0.2)[1])
```

```
37.4400000000000
```

```
print(taylor1(f, x, y, dx, dy, 1, 2, -0.2, 0.2)[1])
```

```
36
```

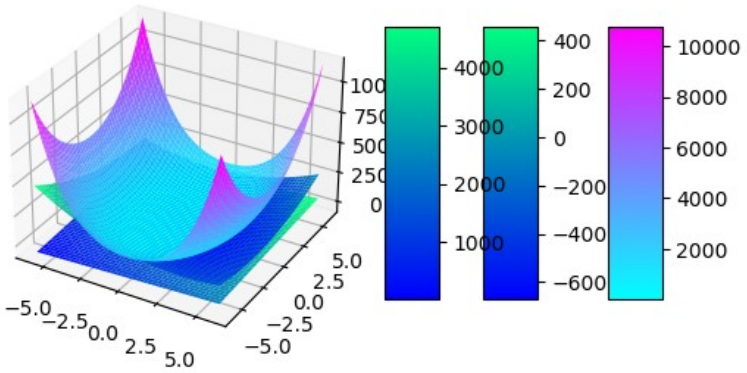
```
(2 * 0.8**2 + 2.2**2)**2
```

```
37.454400000000014
```

в)

```
from matplotlib import cm

x0 = 1
y0 = 2

fig = plt.figure()
xs = np.linspace(-6, 6, 50)
ys = np.linspace(-6, 6, 50)
```

```
X, Y = np.meshgrid(xs, ys)
Z = (2 * X**2 + Y**2)**2

p1 = taylor1(f, x, y, dx, dy, 1, 2, -0.2, 0.2)[0]
ZZ = f.subs([(x, 1), (y, 2)]) + (X - 1) * 8 * x0 * (2 * x0**2 + y0**2)
ZZ += (Y - 2) * 4 * y0 * (2 * x0**2 + y0**2)

ZZZ = ZZ + 1 / 2 * sp.diff(f, x, 2).subs([(x, x0), (y, y0)]) * (X - x0)**2
ZZZ += 1 / 2 * sp.diff(f, y, 2).subs([(x, x0), (y, y0)]) * (Y - y0)**2
ZZZ += sp.diff(sp.diff(f, x), y).subs([(x, x0), (y, y0)]) * (X - x0) * (Y - y0)

fig, ax = plt.subplots(subplot_kw={"projection": "3d"})
surf = ax.plot_surface(X, Y, Z, cmap=cm.cool, linewidth=0)
surf1 = ax.plot_surface(X, Y, ZZ, cmap=cm.winter, linewidth=0)
surf2 = ax.plot_surface(X, Y, ZZZ, cmap=cm.winter, linewidth=0)
fig.colorbar(surf, shrink=0.5, aspect=5)
fig.colorbar(surf1, shrink=0.5, aspect=5)
fig.colorbar(surf2, shrink=0.5, aspect=5)
plt.show()
```



Упражнение 1С. Вычислите якобиан перехода от декартовой системы координат к сферической (переход осуществляется по формулам: $x = r\cos\varphi\cos\theta$, $y = r\sin\varphi\cos\theta$, $z = r\sin\theta$).

```
r, theta, phi = sp.symbols('r theta phi')
x = r * sp.cos(phi) * sp.cos(theta)
y = r * sp.sin(phi) * sp.cos(theta)
z = r * sp.sin(theta)
A = sp.Matrix([[x, y, z]])
B = sp.Matrix([[sp.diff(A, r)], [sp.diff(A, phi)], [sp.diff(A, theta)]])
B
```

$$\begin{bmatrix} \cos(\phi)\cos(\theta) & \sin(\phi)\cos(\theta) & \sin(\theta) \\ -r\sin(\phi)\cos(\theta) & r\cos(\phi)\cos(\theta) & 0 \\ -r\sin(\theta)\cos(\phi) & -r\sin(\phi)\sin(\theta) & r\cos(\theta) \end{bmatrix}$$

```
sp.pprint(sp.simplify(B.det()))
```

```
 2
r ⋅cos(θ)
```

Упражнение 2С.

Создайте **функцию**, раскладывающую функцию $f(x_1,...,x_n)$ в точке $\tilde{M}(\tilde{x}_1,...,\tilde{x}_n)$ по формуле Тейлора до членов 2-го порядка включительно (n - произвольное число). В число входных параметров включите саму функцию f, заданную в символьном виде, координаты точки $\tilde{M}(\tilde{x}_1,...,\tilde{x}_n)$, приращения $\Delta x_1,\ldots,\Delta x_n$. В число выходных параметров включите символьное разложение $p_2(x_1,...,x_n)$ функции $f(x_1,...,x_n)$ по формуле Тейлора до членов 2-го порядка включительно, записанное через приращения $\Delta x_1,\ldots,\Delta x_n$ независимых переменных, а также приближенное значение функции $f(x_1,...,x_n)$ в указанной точке $x_1=\tilde{x}_1+\Delta x_1$, …, $x_n=\tilde{x}_n+\Delta x_n$ (значение $p_2(\tilde{x}_1+\Delta x_1,...,\tilde{x}_n+\Delta x_n)$).

```
def teilor2_n(f, X, Dx, x0, delx):
    sizex = len(X)
    dif1 = 0
    dif2 = 0
    f0 = f
    for i in range(0, sizex):
        dif1 = dif1 + sp.diff(f,X[i]) * Dx[i]
    for i in range(0, sizex):
        f0 = f0.subs(X[i], x0[i])
    for i in range(0, sizex):
        for j in range(0, sizex):
            df2 = sp.diff(sp.diff(f, X[i]), X[j]) * Dx[i] * Dx[j]
            if (i!=j):
                df2 = df2 * 2
            dif2 = dif2 + df2
    df1subs = dif1
    df2subs = dif2
    for i in range(0, sizex):
        df1subs = df1subs.subs(X[i], x0[i])
        df2subs = df2subs.subs(X[i], x0[i])
    for i in range(0, sizex):
        df1subs = df1subs.subs(Dx[i], delx[i])
        df2subs = df2subs.subs(Dx[i], delx[i])

    p1= f0 + df1subs + 0.5 * df2subs
    p1sim = f + dif1 + 0.5 * dif2
    print(p1sim)
    print(p1)
    return p1sim
```

```
x, y, z, w, dx, dy, dz, dw = sp.symbols('x y z w dx dy dz dw')
f = x**2 - w * (sp.sqrt(y) - z) + z * x
X = [x, y, z, w]
x0 = [2, -1, 4, -3]
Dx = [dx, dy, dz, dw]
delx = [0.1, -0.2, 0.5, -0.2]
teilor2_n(f, X, Dx, x0, delx)
```

```
-1.0*dw*dy/sqrt(y) + 2.0*dw*dz + dw*(-sqrt(y) + z) + 1.0*dx**2 + 2.0*dx*dz + dx*(2*x + z) +
0.125*dy**2*w/y**(3/2) - dy*w/(2*sqrt(y)) + dz*(w + x) - w*(sqrt(y) - z) + x**2 + x*z
-0.59 + 3.525*I
```

$$-\frac{1.0dwdy}{\sqrt{y}} + 2.0dwdz + dw\left(-\sqrt{y} + z\right) + 1.0dx^2 + 2.0dxdz + dx\left(2x + z\right) + \frac{0.125dy^2w}{y^{\frac{3}{2}}} - \frac{dyw}{2\sqrt{y}} + dz\left(w + x\right)$$
$$- w\left(\sqrt{y} - z\right) + x^2 + xz$$